PAALHU: remaining work for Practice 1

Web Application Programming (G247) · Hugo Mateos, Pablo Cuenllas, Álvaro Morillo · updated 7 November 2026 · Repository: `github.com/hugomateos1/PAALHU` (branch `main`)

> This is the only planning document. The two older plans of 29 October (`PAALHU-controllers-pending` and `PAALHU-pending-work`) were deleted because they did not follow the Practice 1 PDF: login is not required, every resource needs full CRUD with its own router, controller and validator, and edits use `PUT`. They remain in the Git history (commit 2bba677).

1. Where we are

Step	What	Status
1	README in the PDF template: authors and the 8 resources with every field and rule	**Done** commit 5691d7d
2	Front end in `public/` (pages/, css/, js/, img/), one `express.static`, JSON 404 for every unknown URL	**Done** commit 5691d7d
3	`dishes` (main resource): store, validator, controller, router, full CRUD	**Done** commit b664d4f
4	The other 7 resources with full CRUD: drinks, setMenus, tables, customers, reservations, orders, reviews	**Done** commit b664d4f
—	Spice symbols (🌿 / 🌶️ × level) on the spice guide, menu and set menu, same scale as `spiceLevel`	**Done** commit 5ba721f
5	Filter and stats on `dishes`	**To do**
6	Front end connected to the API with `fetch()`. **Done:** `api.js`, `forms.js`, `crud.js`, booking form, staff pages for reservations, orders, set menus and drinks. **Left:** menu, set menu, home (reviews + stats), staff pages for tables, customers and reviews.	**In progress** Álvaro, 7 Nov
7	Postman collection, README completed, AI report. **Done:** README examples, front-end table (pages done so far) and Design decisions. **Left:** Postman, README Postman section and filter/stats examples, AI report.	**In progress**

What the API has now

8 resources, each under `/api/<resource>` with `GET /`, `GET /:id`, `POST /`, `PUT /:id`, `DELETE /:id`. All were tested: 200 · 201 + `Location` · 204 · 400 `{ errors: [{ field, message }] }` with every error at once · 404 for an unknown id (also for an id that is not a number). Unknown fields and any `id` sent by the client are ignored.

Layer	Where	Job
Router	`src/routes/<resource>.js`	Only maps method + path to a controller function
Controller	`src/controllers/<resource>Controller.js`	Reads `req`, calls the validator and the store, answers with the status code
Validator	`src/validators/<resource>Validator.js` (+ `common.js`)	Checks the body, returns `{ valid, errors, value }`; never touches data
Store	`src/data/store.js`	The only file that touches the data (in memory, reset on every restart)

The booking chat (`/api/chat`, `public/js/chat.js`) and the assistant skills in `.claude/` are extra: the PDF does not ask for them, but the team must be able to explain them in the defence.

2. Step 5 · Filter and stats on dishes

GET /api/dishes with query parameters	
Reads	`?search=&category=&vegetarian=&available=`. All optional, and they combine: `?search=naan&vegetarian=true`.
Does	`search`: case-insensitive match on `name`. `category`: one of starters, curries, vegetarian, rice, breads, desserts. `vegetarian` / `available`: only the text `true` or `false` (query values always arrive as text).
Answers	200 with an array (possibly empty) · 400 `{ errors: [{ field, message }] }` for a category outside the list or a flag that is not true/false.
Where	A query validator (e.g. `validateDishQuery(query)` in `dishesValidator.js`), the filtering in `listDishes` of the controller. Filtering happens on the server.

GET /api/dishes/stats	
Answers	200 `{ total, vegetarian, available, averagePrice, byCategory: { starters: 5, curries: 7, … } }`, calculated on every request, never stored.
Watch out	In `src/routes/dishes.js` it must be declared **before** `/:id` (the comment marks the place); otherwise Express reads "stats" as an id.

3. Step 6 · Connect the pages to the API

Keep the HTML structure, classes and stylesheet; replace the data typed into the HTML with data from the API. Open the site only at `http://localhost:3000` (`npm start`), never by double-clicking.

1. `public/js/api.js` with `apiRequest(method, path, body)`: the ONE place that calls `fetch`. It must check `response.ok` itself (fetch does not fail on 404 or 500) and return the data or the error list.
2. **One script per page** in `public/js/`: find the element, ask the API, build one element per item with `createElement` + `textContent`, show them with `replaceChildren`.
3. Every page must show a **loading**, an **error** and an **empty** state.

Already done (7 November, Álvaro)

File	What it does
`public/js/api.js`	`apiRequest(method, path, body)`, the only `fetch`. Always returns `{ ok, status, data, errors }`; `errors` is always a list of `{ field, message }` (also for a 404, a 500 or the server being down). **Use it on every page; do not write another one.**
`public/js/forms.js`	`readForm` (checkbox → true/false, number input → number), `fillForm`, `checkForm` (client-side check from the HTML attributes and `data-message`), `showErrors` (each message in `<span class="error" data-for="field">`), `euros`, `yesNo`.
`public/js/crud.js`	`setupManager({ resource, singular, columns, check })`: list with loading / error / empty states, create (POST), Edit (PUT), Delete (DELETE), list reloaded after each change.

`pages/booking.html` + `js/booking.js`	Customer reservation form → `POST /api/reservations` (status always `confirmed`, no day before today). The chat information is kept under the form.
`pages/manage.html`	Staff home, linked from the footer of the home page.
`pages/manage-{reservations,orders,set-menus,drinks}.html` + `js/manage-*.js`	One staff page per resource. The `<select>` lists copy the validators' constants (a comment in the HTML says which). Orders: address required only for delivery, also checked in the browser.
`css/styles.css`	New rules at the end: forms, error messages, status messages, staff tables, small buttons, `[hidden]`.

> **Pablo, staff pages for tables, customers and reviews:** copy `manage-drinks.html` and change the form fields (same rules as the validator, message in `data-message`); the script is only a `setupManager({...})` call with the columns, like `manage-drinks.js`. Then add the page to the nav of every staff page and to `manage.html` (a comment marks the place).

Tested on a fresh server in headless Chrome: lists drawn from the API, create / edit / delete, client-side errors, the server's 400 under its field, delivery without an address, a `+34` phone, and the booking form.

Still to do

Must have (PDF)	Suggestion for PAALHU
Every resource listed from its GET, no hard-coded rows left	`menu.html`: dishes and drinks (chillies drawn from `spiceLevel`, so levels live only in `store.js`). `set-menu.html`: setMenus. Home: reviews.
Create (POST), edit (PUT) and delete (DELETE) for **every** resource, list updates without reloading	Done reservations, orders, setMenus, drinks. **Left:** staff pages for tables, customers and reviews (Pablo, with `setupManager`) and for dishes.
On a 400, each message next to its field; a client-side check first	Done in `forms.js`; new pages get it by using `showErrors` / `setupManager`.
Search box + at least one `<select>` or checkbox building the query string	On `menu.html`: search box, category `<select>`, "vegetarian" checkbox → `GET /api/dishes?…`.
Stats from `/api/dishes/stats` somewhere	A small summary on the home page.
Fixed `<select>` lists with exactly the validator's values	Done drink type, service, reservation status, order type and status. **Left:** dish category (menu filter and dish staff page).

4. Step 7 · Finish

- **Postman:** export (Collection v2.1) to `postman/paalhu.postman_collection.json`. A collection variable `baseUrl = http://localhost:3000/api`; at least one full CRUD (list, get one, create, replace, delete) that runs in order on a fresh server (create first, then use the new id). Recommended: one invalid POST (400), the filter and `/stats`.
- **README:** Done examples for the five CRUD endpoints (in full for `drinks`, one body per other resource, one 400), the "Front end" section and Design decisions. **Left:** examples for the filter and `/stats` (after step 5), how to import and run the Postman collection, and the rows of the front-end table for the pages still to connect.
- **AI report** `AI/ai-report.docx`, 2 pages max, every member named: how AI was used (what was kept, changed or rejected, and why), advantages with real examples, one good prompt quoted in full with its weaker first version. Every claim needs a screenshot or a shared-conversation link.

5. Open decisions and small things

Topic	What to do
Spice levels	The per-dish levels in `store.js` are guesses (e.g. vindaloo 5, madras 4, korma 1). Check them; until step 6 also change `menu.html`, `set-menu.html` and the assistant's `reference.md`. Mulligatawny soup (set menu only) is level 2.
Invalid query parameters	**Decided** 400, reason written in the README's Design decisions. Step 5 must implement it that way.
Design decisions	**Written** in the README: pages in `public/pages/` · where `express.static` is mounted · non-numeric id → 404 · search and filter on the server · invalid query → 400 · validation on both sides · `<select>` lists copy the validators · reservations validate the format only · separate staff pages, no login · data in memory. Add any new decision there.
Table codes	Two tables can share a code (e.g. two M7). Optional: reject a repeated code with 409.
Data in memory	Everything created through the API is lost when the server restarts. Expected for Practice 1; the optional +0.5 bonus is a real SQLite or MongoDB behind the same `store.js` functions.

6. Suggested split

The PDF wants a commit history showing **every member contributing**, and in the defence (4 of the 10 points) each member must explain any file, including the others'. Steps 1–4 were committed by Hugo, so Pablo and Álvaro should commit the next parts.

Still unassigned: `set-menu.html` from `GET /api/setMenus`, and a staff page for dishes (main resource, needs the category `<select>`).

Member	Suggested work
Hugo	Step 5 (filter + stats) · the menu page (dishes, drinks, search and filter) · stats on the home page · README examples for the filter and `/stats`. (`api.js` was already written by Álvaro: use it.)
Pablo	Staff pages for tables, customers and reviews (with `setupManager`, see section 3) · reviews on the home page · Postman collection and its README section
Álvaro	**Done** `api.js`, `forms.js`, `crud.js` · booking page · staff pages for reservations, orders, set menus and drinks · README examples, "Front end" section and Design decisions. Free to help with set-menu.html or the dishes staff page.
All	AI report together · everyone reads and can explain the whole `src/` folder

7. Final checklist (grading, section 9 of the PDF)

☐ API (2.5): 8 resources with correct CRUD ✓ · filter and `/stats` work · exact status codes ✓ · routes, controllers and store separated ✓ · unknown routes and errors return JSON ✓

☐ Client (2.0): every resource listed and editable from the pages · fixed `<select>` lists match the validators · search and filter on the server · loading, error and empty states · styled and responsive

☐ Validation & errors (0.5): all messages at once ✓, shown next to their field ✓ (pages done so far) · unknown fields and client ids ignored ✓ · invalid query parameters handled

☐ Postman (0.5): one full CRUD, runs in order on a fresh server, uses `{{baseUrl}}`

☐ Repo & README (0.5): runs from the README alone (`npm install`, `npm start`) · follows the template · every resource and endpoint documented · shared commit history

☐ Defence (4.0): every member can explain every file and decision

☐ AI report: 2 pages max, every member named, every claim with evidence